

Introducción a la API REST de GitHub

La API REST de GitHub permite interactuar con GitHub mediante código (crear issues, obtener repositorios, automatizar tareas). Este documento explica cómo usarla con GitHub CLI y JavaScript.

Pasos básicos para usar la API

Paso	Qué hacer
1	Selecciona un endpoint (punto final de la API)
2	Identifica el método HTTP (GET, POST, etc.) y la ruta
3	Reemplaza los parámetros de ruta {owner}, {repo}, etc.
4	Envía la solicitud usando GitHub CLI o JavaScript
5	Revisa la respuesta (código, encabezados, cuerpo)

Ejemplo de endpoint "Crear un issue":

- Método: POST
- Ruta: /repos/{owner}/{repo}/issues
- Parámetros: {owner} = dueño del repo, {repo} = nombre del repo

Usar la API con GitHub CLI

Comando base:

```
gh api --method <MÉTODO> <RUTA> --header <ENCABEZADOS> -f <PARÁMETROS>
```

Opciones principales:

Opción	Uso
--method	Método HTTP y ruta (ej. GET /octocat)
--header	Encabezados como Accept o X-GitHub-API-Version
-f	Parámetros de tipo cadena (clave=valor)
-F	Parámetros numéricos, booleanos o nulos
--include	Muestra también el código de estado y encabezados

Encabezados obligatorios:

Encabezado	Ejemplo
Accept	application/vnd.github+json
X-GitHub-API-Version	2022-11-28

Ejemplos prácticos

Obtener Octocat en ASCII:

```
gh api --method GET /octocat \
  --header 'Accept: application/vnd.github+json' \
  --header "X-GitHub-API-Version: 2022-11-28"
```

Listar solo 2 eventos:

```
gh api --method GET /events -F per_page=2 -F page=1 \
  --header 'Accept: application/vnd.github+json'
```

Crear un issue:

```
gh api --method POST /repos/octocat/Spoon-Knife/issues \
  --header "Accept: application/vnd.github+json" \
  --header "X-GitHub-API-Version: 2022-11-28" \
  -f title='Creado con la API REST' \
  -f body='Issue de prueba'
```

Para arrays en parámetros:

```
gh api ... -f repository_ids[]=ID1 -f repository_ids[]=ID2
```

Entendiendo la respuesta

La API devuelve tres componentes:

Componente	Qué es
Código HTTP	200 = éxito, 404 = no encontrado, 401 = no autorizado, etc.
Encabezados	Incluyen límites de uso, fechas, etc.
Cuerpo (body)	Datos en formato JSON (campos vacíos = null, horas en UTC)

Encabezados importantes de GitHub:

Encabezado	Qué indica
X-Ratelimit-Limit	Máximo de solicitudes permitidas
X-Ratelimit-Remaining	Solicitudes restantes
X-Ratelimit-Reset	Cuándo se reinicia el límite
Link	Enlaces de paginación (siguiente, anterior, última)

Ver código y encabezados (--include):

```
gh api --method GET /repos/octocat/Spoon-Knife/issues -F per_page=2
--include
```

Guardar respuesta en archivo y filtrar con jq:

```
gh api /repos/octocat/Spoon-Knife/issues > data.json
jq '.[0] | {titulo: .title, autorID: .user.id}' data.json
```

Representaciones: detallada vs resumida

Tipo	Cuándo	Qué incluye
Detallada	Obtener un solo recurso	Todos los atributos
Resumida	Obtener una lista	Solo atributos principales

Los atributos costosos computacionalmente se omiten en la versión resumida.

Hipermedia

Los recursos suelen incluir propiedades que terminan en `_url` (ej. `html_url`). Úsalas en lugar de generar las URL manualmente. Esto facilita actualizaciones futuras de la API.

Límites de velocidad (Rate Limiting)

GitHub limita cuántas solicitudes puedes hacer por tiempo. Monitorea estos encabezados:

Encabezado	Qué indica
------------	------------

X-Ratelimit-Limit	Máximo permitido
X-Ratelimit-Remaining	Cuántas quedan
X-Ratelimit-Reset	Momento del reinicio

GitHub Apps (automatización avanzada)

Son aplicaciones que extienden GitHub. Pueden:

- Actuar en nombre de un usuario (requiere autorización del usuario)
- Actuar por sí mismas (sin intervención del usuario)
- Responder a webhooks (eventos que ocurren en GitHub)

Tipos de autenticación:

Comportamiento	Token usado
En nombre del usuario	Token de acceso de usuario
Por sí misma	Token de acceso de instalación

Casos de uso comunes:

Tipo	Ejemplo
En nombre del usuario	"Iniciar sesión con GitHub"
Por sí misma	CI/CD, automatización al recibir un webhook

Pasos para crear una GitHub App:

1. Registrar la app (permisos, webhooks)
2. Escribir el código
3. Instalar la app en una cuenta personal u organización
4. (Opcional) Compartirla mediante enlace o GitHub Marketplace

Buenas prácticas:

Práctica	Por qué
Mantener credenciales seguras	Evita accesos no autorizados
Solicitar mínimos permisos	Reduce riesgos
Usar state en URL de instalación	Para mantener el estado del usuario

Compartir una GitHub App:

5. Ve a Developer settings → GitHub Apps → Editar

6. Haz clic en Página pública → Instalar
7. Copia la URL: <https://github.com/apps/NOMBRE-DE-TU-APP/installations/new>
8. Puedes agregar `?state=TU_VALOR` a la URL para preservar estado

Resumen rápido para empezar

Recomendación	Por qué
Usa gh api para probar	Rápido desde la terminal
Incluye siempre Accept y X-GitHub-API-Version	Obligatorio para que funcione
Reemplaza {owner}, {repo}, etc.	Son marcadores obligatorios
Monitorea X-Ratelimit-*	Para no exceder los límites
Usa --include para depurar	Ves código y encabezados
Para automatización seria, usa GitHub App	Más segura y flexible que tokens personales
Usa jq para procesar JSON	Extrae solo lo que necesitas